

LJA -- Pipeline

Learning Journey Assistant — CLI Pipeline

What `python -m lja.cli` does

The commands that read what it wrote

Reading the diagram

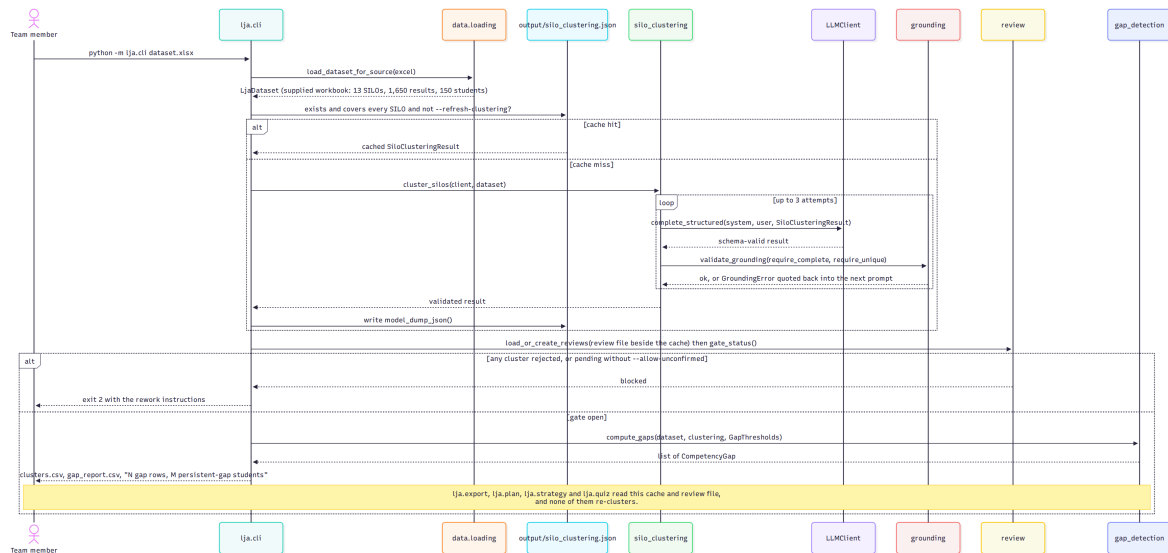
Timings on the supplied workbook

Learning Journey Assistant — CLI Pipeline

Version	2.0, 3 October 2026. Supersedes the 12 August 2026 page.
Describes	main at b4ccebdc (3 Oct 2026)
Source	docs/design/cli-pipeline.md; diagram from docs/handover/diagrams/04-seq-pipeline.mmd. Rebuild with docs/design/build.sh

What `python -m lja.cli` does

One command turns a workbook, or a Moodle database, into competency clusters and a per-student gap report. It has exactly one step that touches a model, and that step is cached so a normal re-run never re-spends the call.



Running the pipeline

Figure 1: Load, cluster (cached), gate, classify. Source: `docs/handover/diagrams/04-seq-pipeline.mmd`.

Step	Where	What happens	Exit
Load	<code>lja.data.loading</code>	--source excel reads the three workbook sheets; --source moodle runs SQL Query 2 as <code>lja_reader</code> and joins the criterion-to-SILO CSV. Both return one <code>LjaDataset</code> .	2 on a malformed workbook or an unmapped criterion
Cluster, or read the cache	<code>lja.model.silo_clustering</code>	If <code>output/silo_clustering.json</code> (or --clustering-cache) exists and covers every SILO, no model call. Otherwise one structured call; the result must place every SILO in exactly one cluster, and a miss is quoted back into a retry, up to three attempts.	1 if no attempt passes coverage
Gate	<code>lja.review</code>	The review file beside the cache holds pending / confirmed / rejected per cluster. Any rejected cluster stops the run and prints the reviewer's note as rework instructions. Pending clusters stop it too unless --allow-unconfirmed.	2 when blocked
Classify	<code>lja.model.gap_detection</code>	For every student and competency: weighted attainment, then floor, ceiling, flat-profile fallback, relative position against the student's own median and MAD. Persistent when two or more subjects evidence the gap.	
Write		<code>output/clusters.csv</code> (one row per SILO, then the SILO definitions) and <code>output/gap_report.csv</code> (student, competency, attainment, subjects evidencing, observations, classification, basis, relative position).	0

Every threshold is an environment variable with a CLI flag (`--absolute-floor`, `--relative-gap-cutoff`, `--min-spread`, ...) so a sensitivity sweep is a loop in a shell, not a code change. The values are proposals, not ratified (action A-01); the dashboard's Provenance page shows the ones in force.

The commands that read what it wrote

None of these re-clusters. Each reads the cache and the review file the pipeline left, and each is run deliberately, because the generating ones cost model calls.

Command	Produces	Model calls
<code>python -m lja.review</code>	Confirm or reject clusters; a rejection needs a note	none
<code>python -m lja.plan <xlsx> STUxxxx</code>	A learning plan: what to work on, with evidence and actions, grounded name by name	1 per attempt, up to 3
<code>python -m lja.strategy <xlsx> STUxxxx</code>	A study strategy: how to study each gap, techniques from a closed evidence-based list, grounded per competency	1 per attempt
<code>python -m lja.quiz <xlsx> STUxxxx</code>	A practice quiz per gap, multiple choice or written task, plus a blind educator review; the answer key is unverifiable and the page says so	1 per attempt + 1 review
<code>python -m lja.export <xlsx> --out DIR</code>	<code>students.csv</code> , <code>competencies.csv</code> , <code>cohort.csv</code> , <code>manifest.json</code> ; <code>--anonymise</code> pseudonymises ids with a keyed HMAC	none
<code>python -m lja.dashboard</code>	Serves the run; classifies at start-up from the workbook and cache	none on a page load; the opt-in Generate button runs <code>lja.plan</code> / <code>lja.quiz</code> as a subprocess

Reading the diagram

Cache hit or miss. Checked before anything else runs. If the cache exists, covers every SILO in the dataset and `--refresh-clustering` was not passed, the model branch is skipped and the run is free. Adding a subject with new SILOs makes the cache stale and triggers a re-cluster; existing review decisions survive for any cluster whose membership is unchanged, because a cluster's id is a hash of its members.

The retry loop. The grounding validator checks the response against the input, not just its shape. A live run in August produced schema-valid JSON that silently dropped three of thirteen SILOs; Pydantic cannot see a completeness problem, only a comparison with the dataset can. The same validator, with different reference lists, is what every generated artefact passes through.

Scale. The single call holds for the three-subject workbook (13 SILOs) and fails on a 52-SILO catalogue on the 30B local model. The handbook tagger (`lja.data.competency_tagger`, embeddings plus k-means with the model labelling clusters in batches) grouped 1,436 real SILOs in 64 seconds and is the staged answer; wiring it into `cluster_silos()` is the next clustering work package.

Timings on the supplied workbook

Load, classify, write (cache hit)	under 2 s
First clustering call, qwen3-v1:30b via Ollama	minutes; run-to-run variance is why the cache and the review file exist
One learning plan, qwen/qwen3-v1-30b via LM Studio	17 to 35 s
Test suite, offline	365 tests, about 3.5 s